

Computer Control · Vetto AI

Full Onboarding Documentation — All Sections

Generated 30 April 2026

PART 1 — General Onboarding

1.1 About Vetto

Welcome — you have been selected to contribute to Computer Control, one of Vetto's core AI evaluation projects. This onboarding covers everything you need to know before your first task — from how Vetto works to how you'll be paid and where you'll do the actual work.

Your Status

You are engaged as an independent contractor, not an employee. Work is project-based. Vetto may assign, pause, or terminate access to projects at any time.

Confidentiality — Critical

NEVER share, post, or discuss any project content — prompts, tasks, datasets, outputs, screenshots, or recordings. Never mention clients, internal tools, or workflows publicly. Any breach results in immediate removal and potential legal action.

You MAY describe your work at a high level (e.g., "AI data annotation contractor") without revealing any sensitive or identifiable project information.

Conduct

Vetto operates a zero-tolerance policy for disrespectful communication, disruptive behavior, harassment, or non-compliance with team instructions.

Security & Usage Policy

Platform Integrity: You must not attempt to exploit, bypass, or interfere with the platform infrastructure in any way.

Sandbox Environment: You are strictly prohibited from attempting to access other users' environments or escaping your assigned sandbox.

Responsible Behavior: Any attempt to exploit the system outside of reporting bugs will result in immediate removal from the project.

Bug Reporting Exception: If you identify vulnerabilities or issues, they should only be used for reporting purposes and shared with the team.

API Usage Policy

API Key Restrictions: API keys provided are strictly for use within the platform environment.

No External Usage: You must not copy, reuse, or use the API key outside of the platform for personal or external purposes.

Budget Awareness: Each task has a limited API budget (\$30 per task, no reset). You are responsible for managing your usage accordingly.

1.2 Communication

Official Channel — Slack

All communication for this project will happen via Slack. Decisions, updates, announcements, and project guidelines are managed there. Slack is the single source of truth for this project.

Your Slack invitation will be sent by email at the end of this onboarding. Check your inbox to access the workspace.

Questions about Slack access

Contact Silas Caetano at silas@vetto.ai if you have any questions about joining the workspace.

Ground Rules

- Do not send project content via personal channels outside the designated Slack channel.
- Keep messages concise and professional.
- When in doubt, ask in the channel before acting.

1.3 Payments

Compensation

R\$ 1,500.00 per approved task. Tasks vary in complexity — expect to invest a solid number of hours across the full cycle from setup to submission. Plan accordingly and work at your own pace.

What Gets Paid

Only tasks that complete the full pipeline and are marked "Approved" in the platform enter the payment cycle. Tasks under review, pending correction, or rejected are not compensated.

Weekly Payment Cycle

- Tasks approved by Tuesday 11:59 PM (BRT)
- Consolidated on Wednesday
- Payment submitted on Friday of the same week

Tasks approved after Tuesday 11:59 PM move to the following week's cycle.

How You Receive Payment

Vetto is incorporated in the United States (Vetto Inc., San Francisco). Payments are delivered in BRL (Brazilian Real) directly to your Brazilian bank account.

Financial Onboarding — Mandatory Before First Payment

Brazil and Euro Zone — Avenia: You'll receive an email invite. Complete KYC verification, submit a W-8BEN form, and add your bank account.

U.S. and Rest of World — Rippling: You'll receive a Rippling invite for identity verification and bank setup.

Complete financial onboarding immediately when you receive the invitation — delays in onboarding delay your payment.

1.4 Where You Do the Work

All task work is done inside Arena (arena.vetto.ai) — Vetto's task platform. You will access it after completing this onboarding.

You do not need to create an account or take any action there right now. Access will be provided at the end of this onboarding, in the Access & Activation step.

PART 2 — Technical Documentation

2.1 Introduction to the Project

What You're Building

You are contributing to a terminal-based AI evaluation benchmark — a dataset of tasks designed to test how well AI agents perform real, grounded work inside a terminal environment. The output feeds directly into model training pipelines: fine-tuning, RLHF, and reinforcement learning from supervised signals.

Each task is a scenario where an AI agent must take actions — reading files, querying databases, calling APIs, manipulating data — and succeed or fail based on objective, verifiable outcomes.

Why This Matters

The quality of the benchmark determines the quality of model improvements downstream. A poorly designed task produces noisy training signal. A well-designed task teaches the model something real. Every task you contribute is a direct input to how a frontier model learns to behave.

Your Role

You are not just writing prompts. You are building evaluation infrastructure. You are responsible for generating, validating, and calibrating tasks that meet the project quality bar.

One rule above all: The task must always have a reachable solution. Difficulty is encouraged. Impossibility is not.

2.2 The Project Pipeline — 6 Stages

Think of yourself as a game level designer. The "players" are AI models. The "levels" are tasks. Your job is to design scenarios that are fair, verifiable, and genuinely hard — so the models that train on them actually improve.

1. Choose or Create a Task — Define the challenge the AI will have to solve.
2. Review & Materialize — Review the AI-generated spec and build the environment.
3. The Oracle — Confirm the task has a real, reachable correct solution.
4. Evaluate & Analyze — Run the task against real models and read results.
5. Harden — Improve difficulty legitimately based on what you observed.
6. Final Review & Submit — Final pass and submit for review and compensation.

2.3 Step 1 of 6 — Choose or Create a Task

You can browse existing task categories on the platform and choose a scenario, or create your own from scratch. There are no restrictions on topic — what matters is that there is a real challenge at the center of it.

The 3 Required Components

1. **Task Title:** A short descriptive name that identifies the scenario and the tools involved. Specific enough that someone reading it knows immediately what the challenge is about. Example: "Reschedule conflicting calendar events using stale email thread"
2. **Task Description:** A narrative written from the agent's perspective. Must include: the trigger (what initiates the task), the environment (what tools are available), the complexity (what conflict or constraint to resolve), and the expected actions (what the agent needs to do).
3. **Verification Conditions:** Objective, deterministic assertions that confirm success. Must cover: positive checks (what must exist), negative checks (what must not have been touched), artifact checks (specific outputs the agent must produce), and quality check (solution was better than a naive alternative).

The 5 Quality Criteria — Every Task Must Pass All Five

If this stage is wrong, every stage that follows is wasted. A task with a flawed goal, an unverifiable outcome, or an impossible solution will fail downstream. Do not rush this.

1. Clear and unambiguous goal — The task must state what the agent needs to accomplish, what tools or environment it has access to, and what the expected end state looks like. All three must be explicit — nothing implied.
2. Verifiable outcome — Success must be checkable through concrete assertions — file state, database entries, outbox content, calendar events. If success depends on interpretation, it is not verifiable.
3. Appropriate difficulty — The task must require the agent to resolve genuine ambiguity or conflict. Avoid tasks where the correct action is obvious from the first line (too easy) and tasks where there is no viable solution (unfair).
4. Single focus — The task should test one coherent capability. Multi-step is fine — multi-purpose is not.
5. Reproducible environment — The task must define a mock environment with a fixed starting state. Verification logic must be deterministic. The same task run twice should produce the same result.

2.4 Step 2 of 6 — Review & Materialize

After you submit your task concept, the platform generates three blocks using AI. Your job is to review what was generated, make sure everything is correct, and click Materialize to turn it into a real runnable environment.

The 3 Generated Blocks (The Spec)

Task Definition: What the evaluated agent sees — the problem statement. No hints about the solution.

Environment Description: What's in the container: OS, services, data files, faults injected, red herrings.

Test Descriptions: Success conditions — each one becomes a verification assertion.

What You Need to Review

Task Definition: Read it as if you were the AI. Is the goal unambiguous? Any hints that make the answer too obvious? Any constraints that make it impossible?

Environment Description: Check that everything the agent needs is present. Red herrings are acceptable — but they must be plausible, not misleading in a way that makes the task unfair.

Test Descriptions: Each condition must map to a clear, verifiable outcome. There must be only one correct interpretation of each assertion.

The Materialize Button

When everything looks correct, click Materialize. This builds the full environment — containers, file structure, mock data, and verification scripts. Materializing does not lock the task permanently, but every re-materialize costs time and API budget. Get the spec right before clicking.

2.5 Step 3 of 6 — The Oracle

Before any model touches your task, you need to confirm it actually has a correct solution. The Oracle runs the golden solution (solve.sh) automatically against the task environment. If it fails, fix the task before running any models.

When the Oracle Passes

The environment was built correctly. A correct solution path exists. The verification logic works. The task is ready for model evaluation.

When the Oracle Fails — Common Reasons

1. The environment is missing something — A required file, database entry, or tool is not present. Go back to the Spec, check the Environment Description, and re-materialize.
2. The golden solution is wrong — The solve.sh script does not actually solve the task correctly. Open the Code Editor, inspect the solution file, and fix it.
3. The verification logic has a bug — The test assertions are checking the wrong thing or are too strict. Inspect the tests folder and fix the verification conditions.
4. The task has no viable solution — The constraints make the task impossible. Go back to the Spec, revise the Task Definition, and re-materialize.

After fixing the issue, re-materialize the task and run the Oracle again. Repeat until it passes.

2.6 Step 4 of 6 — Evaluate & Analyze

The Oracle passed — the task is solvable. Now you run it against real models and read the results. You evaluate, analyze, and use what you learn to decide whether the task needs more hardening or is ready to submit.

The Evaluation Process

When you click Evaluate, the platform runs your task against the selected model in parallel — 4 to 5 independent trials at the same time. This gives you a pass rate rather than a single yes/no, which is statistically much more reliable.

Every evaluation run costs API credits from your \$30 per-task budget. Always start with the Fast model — it is cheaper. Only escalate to the Smart model when the Fast model results are clear and you have confirmed the task is failing for the right reasons.

Model Escalation Order — Never Skip a Tier

1. **Fast model (start here — cheapest):** Should fail (~0% pass rate). If it fails, analyze why before escalating. If it passes easily, the task is too easy — harden more.
2. **Smart model (escalate only when ready):** Target 50-70% fail rate. Some passes, some failures — this is the sweet spot. If it passes 100%, harden more.

Reading the Results — Three Tabs

Trajectory: Every action the model took — what it explored, what commands it ran, where it stopped. Shows you how the model thought about the problem.

Test Results: Which verification checks passed and which failed. Tells you exactly where the model went wrong against the success conditions.

Analysis: AI breakdown of each phase — Exploring, Diagnosing, Implementing, Testing. Saves you from reading every log line.

The Most Important Question After Each Run

When the model fails, do not immediately escalate. Ask yourself: did it fail for the right reason?

Good failure: The model understood the problem, took the right approach, but made a genuine reasoning error. The task is fair, the rules are clear, and there is one correct answer. This is what you want.

Bad failure: The model failed because the task has ambiguity, a hidden hint, or more than one valid answer. This is a task design problem — fix the task, do not escalate.

Other Failure Categories

1. Verifier too strict — The model actually solved the task correctly, but the verification script rejected it. Fix the verifier — this is a false negative.
2. Correct investigation, wrong execution — The model understood the problem and diagnosed it correctly, but produced the wrong output.

Good difficulty — the task is working as intended.

3. Wrong investigation — The model went in the wrong direction. The task may need lighter nudges or clearer framing — check for ambiguity first.
4. Shortcut or mutation — The model bypassed the intended solution. Add explicit negative constraints.

2.7 Step 5 of 6 — Refine

Refining is the most important stage in the project. It is not only about making a task harder — it is about making it right. A task may need refining because the difficulty is off, but also because the instructions are ambiguous, a verification check is broken, a criterion fails, or a red herring is too obvious. Fix whatever is wrong.

Two Hard Rules — Never Break These

The difficulty cannot be artificial. Ambiguous rules, contradictory instructions with no correct answer, information missing with no way to find it — not acceptable. If a human expert could not solve it, it is a broken task.

The prompt can be implicit, but must have a reachable solution. You can make the path hard to find. You cannot make the answer unreachable.

Refining Techniques — In Order of Impact

1. Remove explicit hints — Replace direct instructions with behavioral nudges.
2. Add red herrings — Introduce plausible-but-wrong data that forces the model to validate hypotheses.
3. Require tool discovery — Hide clues behind CLI tools the model must actively find. The model has to read --help output and experiment.
4. Increase data volume and edge cases — Add more rows, more events, DST transitions, anonymous users.
5. Separate investigation from execution — Make finding the problem achievable but producing the exact correct output genuinely hard.
6. Add explicit negative constraints — After red-teaming, add rules that close unintended shortcut paths.
7. Inject real-world noise — Add spam emails, irrelevant events, malformed log entries. The model has to filter noise to find the actual problem.

Exit Criteria — When Refining is Done

Stop refining when ALL of the following are true simultaneously:

- Oracle still passes at 1.000 after the latest version
- Fast model fails consistently — 0% pass rate across all trials
- Smart model fails 50-70% of the time — some passes, some failures
- No artificial difficulty — no ambiguity, no missing information, no impossible constraints
- One correct answer — the model that passes is getting it right, not guessing

2.8 Step 6 of 6 — Final Review & Submit

This is your last checkpoint before the task leaves your hands. Do one final pass through everything — not to redo the work, but to confirm nothing was missed. A task submitted with a hidden problem will not be approved.

Final Checklist — All Six Must Be Checked

1. Oracle passes at 1.000 — The golden solution runs correctly. The task has a real, reachable correct answer. Verify it is still true after the latest version.
2. Fast model fails consistently — Fails 100% of the time across all trials. If it is passing even occasionally, refine more before submitting.
3. Smart model is in the golden zone — Fails 50-70% of trials. Some passes, some failures. If it passes 100% or fails 0%, adjust the difficulty.
4. No hidden hints or unnecessary information — Read the instructions one more time as if you were the AI. Remove anything that makes the answer too obvious.
5. No ambiguity and no artificial difficulty — The task has one defensible correct answer. Difficulty comes from genuine complexity.
6. All tools and information available — Everything required to solve the task is present in the environment. The agent does not need to guess or assume anything.

What Happens After You Submit

1. Cross review — Two independent reviewers evaluate the task and the model response as a pair. If both approve, the task is accepted into the dataset.
2. Third review if needed — If either reviewer raises a concern, the task goes to a third review for a final decision. Handled internally.
3. Task approved — compensation — If the task passes review, it enters the training dataset and you are compensated.

Only approved tasks are compensated. Tasks that fail review at any stage are not paid. This is why the final checklist matters — a task submitted too early is a task that does not get approved.

You created a challenge. You made it fair. You made it hard. You confirmed it has a real answer. Now it goes into the dataset and trains a frontier model. Every task you submit that gets approved makes the model better.

This document was generated from the Vetto AI Computer Control onboarding platform. For the latest version of these guidelines, access the onboarding page directly. All communication and updates are managed via the project Slack channel. Contact: mario@vetto.ai